

Indice

Manuale operativo — claudiocammarano.com	2
Stack in due righe	2
Struttura cartelle	2
Comandi	3
1. Pubblicare un saggio (writing)	4
File	4
Frontmatter minimo	4
La notazione dell'intervento AI — obbligatoria, blocca il build	4
Campi opzionali	5
Tag category — i valori "storici" dei writings	5
Struttura corpo	5
Shortcode disponibili	6
Formule matematiche (LaTeX)	6
PDF ed ePub — automatici, per i saggi da 15 minuti in su	6
Licenze	6
Collegare i concetti — passaggio manuale, non automatico	6
2. Pubblicare un curated	7
File	7
Frontmatter	7
Il criterio di selezione: criterio, perche, rinvio	7
I tags dei curated ora contano come Argomenti — ma solo se sono "riconosciuti"	8
Body	9
3. Pubblicare una nota del Lab	9
Solo in inglese, per scelta	9
URL — /lab/, dall'agosto 2026	9
La cornice del Lab è inglese	9
File	9
Frontmatter	9
Campi opzionali	9
Collegamento ai concetti del sito — tagging manuale e selettivo	10
Tassonomia del Lab — separata da quella del sito	10
Cosa NON fa questo layout (di proposito)	10
Visibilità	10
Documenti citati — link automatico e Learning Log	11
4. Aggiungere una serie	12
5. Tassonomia concetti (_data/conceptsIndex.js)	12
Struttura di una voce	12
Aggiungere un nuovo concetto	12
Tipi validi — attenzione a paese vs luogo	13
Aggancio all'entità reale — il campo sameAs	13
La geografia — il campo geo (obbligatorio da settembre 2026)	13
Legami dichiarati fra concetti — il campo related	14
Il controllo automatico — niente più concetti persi in silenzio	15
Il controllo che blocca — validate-concepts.js	15
L'indice come dataset pubblico	15
6. Come funziona la navigazione semantica	16
Temi (/temi/) e Argomenti (/tag/slug/)	16
Concetti (/concetti/slug/)	16
Il flusso dei dati, in breve	16
Il Sommario ragionato (home)	17
7. Temi e Argomenti (_data/clusters.js)	17
8. La superficie inglese (/en/)	17
Struttura e regola non negoziabile	18
Che cosa esiste in inglese	18
Dichiarare una coppia di pagine equivalenti	18
Le due variabili del layout, da non confondere	18
Regole di contenuto sulle pagine inglesi	18

Aggiungere una pagina inglese	18
Card social	19
9. Gli strumenti interattivi (il banco di misura)	19
Dove vivono	19
Perché il CSS è tutto sotto .bm	19
Il modo incorporato	19
Debito noto — leggere prima di metterci mano	19
10. Cosa NON toccare senza capire	19
11. Troubleshooting rapido	20
12. Linea editoriale e criteri di rifiuto	21
La linea	21
I tre criteri di rifiuto	21
Antagonisti retorici (fonti integrative)	21

Manuale operativo — claudiocammarano.com

Sito statico Eleventy (v3). Sorgenti in `src/`, output compilato in `_site/`. Deploy automatico su Netlify a ogni `git push` sul branch `main`.

Revisione: 28 agosto 2026. Rispetto alla versione di luglio sono cambiati il percorso del Lab, la notazione dell'intervento AI (che blocca il build), la superficie inglese e il modo in cui vivono i due banchi di misura: sezioni 1, 3, 8 e 9.

Stack in due righe

Pezzo	Tecnologia
Generatore	Eleventy 3 + Nunjucks
Stile	CSS custom (<code>src/css/style.css</code>)
Deploy	Netlify (auto da <code>git push</code>)
Grafo concetti	Coordinate calcolate a build time da <code>_data/graphLayout.js</code> , disegno <i>hierarchical edge bundling</i> in <code>src/js/heb.js</code> (JS puro, nessuna dipendenza esterna)
Lingue	Italiano su <code>/</code> , superficie inglese su <code>/en/</code> più tutto <code>/lab/</code> — vedi sezione 8
Font	Source Serif 4, ospitato sul dominio (<code>src/fonts/fonts.css</code> , Fontsource, SIL OFL). Nessuna richiesta a Google Fonts
Formule	MathJax 3.2.2 ospitato in <code>src/vendor/mathjax/</code> , caricato solo dai saggi che lo richiamano
PDF ed ePub	GitHub Action <code>.github/workflows/ebooks.yml</code> + Pandoc/XeLaTeX (<code>scripts/ebooks/</code>) — sezione 1
Ricerca	Pagefind (<code>npm run index</code>)

Struttura cartelle

```
src/
  writings/      # Saggi originali (.md)
  curated/       # Link esterni con commento (.md)
  learning/      # Note di apprendimento (.md)
  lab/           # Note di ricerca in progress, in inglese (.md) —
output su /lab/, vedi sezione 3
```

```

    en/                # Pagine della superficie inglese (.njk) – /en/, /en/start-
here/, /en/about/ – vedi sezione 8
    pages/            # Pagine con permalink esplicito che non stanno nella loro cartella di destinazione: oggi i
vedi sezione 9
    _data/
        clusters.js      # Temi → Argomenti (tassonomia a 5 cluster)
        conceptsIndex.js  # Tassonomia Concetti (fonte di verità)
        curatedTagAliases.js # Tag liberi dei curated → Argomento canonico
        tagDescriptions.json # Descrizioni degli Argomenti (facoltative)
        temiDescriptions.json # Descrizioni dei Temi/cluster (facoltative)
        seriesDescriptions.json
        graphLayout.js    # Calcola il layout del grafo concetti – generato, non a mano
    _includes/
        layouts/
            base.njk      # Layout base (header, footer)
            article.njk   # Layout saggi
            lab-note.njk  # Layout note /lab/ – versione leggera di article.njk
        components/      # Header, footer ecc.
    css/
        style.css        # Unico file CSS
    images/              # Immagini articoli
        og/              # Card social 1200x630, una per pezzo, fatte a mano – richiamate da `og_card`
    js/                  # JS client-side: heb.js (disegno del grafo)
    downloads/          # File scaricabili del Learning Log (risorse + documenti del Lab, vedi sezione 3)
    graph-data.njk      # Endpoint statico /graph-data.json letto dal grafo (generato)
    mappa.njk           # Pagina /mappa/ – grafo completo concetti
    *.njk               # Pagine indice (temi, indice, curated, serie...)

```

Comandi

Sviluppo locale con live reload

```

cd ~/Documents/PROJECTS/claudio-cammarano-site
npm start
# → http://localhost:8080

```

Build di produzione

```

npm run build

```

Deploy

```

git add -A
git commit -m "descrizione"
git push

```

Netlify fa il resto in automatico

Importante: lanciare sempre `npm run build` (o `npm start`) prima di un push importante e leggere l'output in console. Ci sono quattro controlli automatici, e si comportano in modo diverso:

- **Concetti non registrati** — avviso, non blocca (sezione 5).
- **Integrità dell'indice concettuale** — errore, **blocca il build**. Legami verso nomi inesistenti, articoli senza sorgente, why mancanti o troppo lunghi: `validate-concepts.js` si ferma e dice quale voce (sezione 5).
- **Geografia dei concetti** — errore, **blocca il build**. Ogni voce di `conceptsIndex.js` deve avere il campo `geo`, con paesi che esistono in `src/_data/geoPaesi.json`: `validate-geo.js` si ferma e dice quale voce (sezione 5).
- **Notazione dell'intervento AI** — errore, **blocca il build** e quindi il deploy. Se manca `ai_prose` su un pezzo in `writings/`, `curated/` o `lab/`, o se il codice è fuori `enum`, `Eleventy` si ferma e dice quale file (sezione 1).

Da settembre 2026 npm start esegue gli stessi controlli del build (hook prestart), quindi valgono anche in sviluppo.

1. Pubblicare un saggio (writing)

File

Posizione: src/writings/YYYY-MM-DD-slug-italiano.md

Slug: titolo in minuscolo, spazi → trattini, accenti rimossi (è→e, à→a ecc.), max 6 parole.

Frontmatter minimo

```
---
layout: layouts/article.njk
title: "Titolo completo"
date: 2026-06-19
description: "1-2 frasi che funzionano da pitch, non da riassunto."
category: ["AI", "Geopolitica"]
lang: "🇮🇹 Italiano"
tags: [writings]
ai_prose: ED          # obbligatorio, blocca il build se manca – vedi sotto
ai_scope: [FM]        # facoltativo, lista
---
```

La notazione dell'intervento AI — obbligatoria, blocca il build

Dal 23 agosto 2026 ogni pezzo in writings/, curated/ e lab/ dichiara quanta della superficie pubblicata è passata da un modello. Non è una misura del peso intellettuale del contributo: è una dichiarazione di superficie. L'archivio anteriore è stato certificato retroattivamente, quindi **non ci sono eccezioni di data**: un pezzo senza ai_prose fa fallire il build.

ai_prose — valore singolo, obbligatorio:

Codice	Significato
"00"	Nessun uso oltre il correttore ortografico. Fra virgolette : senza, YAML lo legge come il numero 0 e il build si ferma
DL	Dialogo — discussione, ricerca e verifica in conversazione. Il testo è mio dalla prima riga
ED	Editing — testo interamente scritto da me e sottoposto a revisione assistita
WR	Scrittura — parte della prosa pubblicata è stata generata

ai_scope — lista, facoltativa: FM (formalizzazione: formule, modelli, codice, tabelle che entrano nel testo) e TR (traduzione da o verso l'inglese di un testo già mio).

La fonte unica di codici, etichette e descrizioni è src/_data/aiNotation.js: legenda del colophon, tooltip in pagina e messaggi di errore del build leggono tutti da lì. **Aggiungere un codice significa modificare quel file e nient'altro.**

La specifica vive in src/_data/aiNotation.js, ed è l'unica fonte: la legenda del colophon, i title in pagina, i messaggi della validazione di build, la pagina /notazione/ e il file /notazione.json attingono tutti da lì. Aggiungere, togliere o ridefinire un codice significa modificare quel file e alzare spec.version scrivendo una riga di spec.changelog — la specifica è pubblicata con licenza CC BY 4.0, e un numero di versione che cambia senza changelog è un numero che non serve a nessuno.

Divisione dei compiti fra le tre superfici, senza testo duplicato: il **colophon** argomenta (perché la notazione esiste, il rapporto col watermarking, la retrodatazione); **/notazione/** è la specifica citabile e adottabile (codici, forma del codice, perimetro, regole di conformità, changelog, formula di citazione); **/notazione.json** è il vocabolario SKOS per le macchine.

Campi opzionali

```
og_image: "/images/nome-file.avif"           # immagine Open Graph e hero
series: "Nome serie, I"                       # formato esatto con virgola + numero romano
english_version: "https://substack.com/..." # URL versione inglese
serie_totale_prevista: 3                     # solo se la serie ha lunghezza pianificata
og_card: "/images/og/og-slug.png"            # card social 1200x630 dedicata, ha la precedenza su og_image
og_title: "Titolo diverso per la scheda"      # quando la card deve dire più del <title>
translation: { lang: en, url: /en/... }       # solo per pagine con una gemella inglese – vedi sezione 8
```

Tag category — i valori “storici” dei writings

Questi sono i valori usati finora dai saggi (maiuscola compresa). L'elenco completo degli Argomenti del sito — molto più ampio, alimentato anche dai curated — è in `src/_data/clusters.js` (sezione 7).

Epistemologia & AI: AI · Epistemologia · Filosofia · Scrittura · Vibe Coding · Claude

Geopolitica & potere: Democrazia · Geopolitica · Medio Oriente · Mediterraneo · Regimi politici · Scenario Planning · Teoria dei giochi

Editoria & comunicazione: Comunicazione · Dual Use · Formazione · Scienze della Comunicazione

Italia & istituzioni: Bologna · Humanities · Istituzioni · Italia · Politica

Un saggio può in linea di principio usare anche uno qualsiasi degli Argomenti nati dai curated (es. Strategia, Media, Economia) — basta scriverlo esattamente come compare in `clusters.js`.

Struttura corpo

```
<!-- Immagine hero (opzionale) -->
<figure class="article-hero">
  
  <figcaption>Didascalia. © Fonte</figcaption>
</figure>
```

Paragrafo di apertura...

Prima sezione

Testo. ****Asserzioni chiave in grassetto come frasi intere.****

Titoli libri e riviste in **corsivo**.

[Link contestuale](https://url.com) integrato nel testo.

Seconda sezione

Tabella markdown:

```
| Colonna A | Colonna B |
|---|---|
| Valore | Valore |
```

Bibliografia essenziale

Cognome, N. (Anno). **Titolo**. Città: Editore.

Shortcode disponibili

```
{% kicker "ETICHETTA BREVE" %}
```

```
{% pullquote "Citazione lunga in evidenza con bordo sinistro." %}
```

```
{% figure "/images/nome.jpg", "Didascalia figura" %}
```

Info-box (sfondo grigio, bordo blu sinistro):

```
<div class="info-box">
  <p>Contenuto nota.</p>
</div>
```

Formule matematiche (LaTeX)

Inserire prima del corpo dell'articolo:

```
<script>
window.MathJax = {
  tex: { inlineMath: [['$', '$']], displayMath: [['$$', '$$']] },
  svg: { fontCache: 'global' }
};
</script>
<script id="MathJax-script" async src="/vendor/mathjax/tex-mml-ctml.js"></script>
```

Poi nel testo: $\$formula$ inline\$ oppure $$$formula$ display\$.

Da settembre 2026 MathJax è servito dal dominio, non più da jsDelivr: non usare l'indirizzo esterno. Nei quattro saggi con formule il percorso è già stato cambiato.

PDF ed ePub — automatici, per i saggi da 15 minuti in su

Non c'è niente da fare a mano. Quando un saggio da almeno 3000 parole (15 minuti a 200 parole al minuto) arriva su main, la GitHub Action «**PDF ed ePub dei saggi**» lo converte con Pandoc (PDF A4 in Source Serif 4 con XeLaTeX, ePub con le formule in MathML), salva i file in `src/downloads/saggi/` con un commit proprio, e Netlify li pubblica. Rigenera solo i saggi cambiati: gli hash stanno in `src/downloads/saggi/manifest.json`.

- Nella pagina del saggio compare «Scarica: PDF · ePub» accanto al tempo di lettura e in coda al testo, **solo se i file esistono** (filtro ebook in `.eleventy.js`).
- In coda a ogni file: codice dell'intervento AI con le etichette per esteso, nota di retrodatazione per i pezzi anteriori al 23 agosto 2026, licenza CC BY-NC-ND 4.0, URL canonico. La licenza è anche nei metadati del file.
- Le parti interattive restano solo online; le immagini WebP e SVG non entrano nel PDF (LaTeX non le gestisce).
- Per rigenerare tutto (per esempio dopo aver cambiato lo script): su GitHub, Actions → «PDF ed ePub dei saggi» → Run workflow, spuntando *Rigenera tutti i file*. In locale: `python3 scripts/ebooks/build.py --force` (servono Pandoc, XeLaTeX e Source Serif 4 installato).
- Requisito una tantum: nel repo, Settings → Actions → General → Workflow permissions → *Read and write permissions*.

Licenze

Saggi: **CC BY-NC-ND 4.0** (dichiarata nel Colophon, nel JSON-LD di ogni saggio e nei PDF/ePub). Learning Log: CC BY-NC-SA 4.0. Dataset dei concetti: CC BY 4.0.

Collegare i concetti — passaggio manuale, non automatico

A differenza dei curated, per i **writings** il collegamento ai concetti non passa dal frontmatter del file: vive in `src/_data/conceptsIndex.js`. **Salvare il file .md non basta.**

- Se l'articolo cita un concetto già in tassonomia → aggiungi { title: "Titolo articolo", url: "/writings/slug-articolo/" } all'array articles della voce corrispondente in conceptsIndex.js (sezione 5 per il formato).
- Se introduce un concetto nuovo → crea una nuova voce.

La skill pubblica-articolo prepara il blocco da incollare in conceptsIndex.js — righe per le voci esistenti e voci nuove complete di geo e, se esiste, sameAs — ma non lo scrive nel file: l'incollatura resta un passaggio tuo, articolo per articolo. Controlla sempre, nella nota finale della skill, le scelte di geografia e Wikidata delle voci nuove.

Checklist completa per pubblicare un writing:

1. Salva il file in src/writings/
2. Apri src/_data/conceptsIndex.js e aggiorna le voci dei concetti citati (vedi sopra). **Ogni voce nuova con il campo geo** (sezione 5), altrimenti il build si ferma
3. npm run build in locale — leggi l'output: se un concetto non è registrato, ora il build te lo segnala (sezione 5)
4. git push — non esiste un comando separato di “aggiornamento semantico”: indici, pagine /concetti/, chip e grafo si rigenerano da soli al build, una volta che i dati sono corretti

2. Pubblicare un curated

File

Posizione: src/curated/YYYY-MM-DD-slug-fonte-tema.md

Convenzione slug: include spesso nome fonte + tema, es. 2026-04-09-nyt-google-ai-overviews.md

Frontmatter

```
---
title: "Titolo originale dell'articolo esterno"
external_url: "https://url-completo.com/articolo"
source: "Nome Autore / Nome Pubblicazione"
date: 2026-06-19
criterio: lettura-di-mercato
perche: "Una frase: la ragione d'archivio condensata, mostrata sotto il badge."
description: "2-4 frasi in prima persona. Non riassunto neutro: commento di merito con punto di vista"
tags: [curated, ai, geopolitica]
concepts: ["Anthropic", "allineamento AI"]
---
```

tags: sempre inizia con curated, poi 1-3 tag tematici in minuscolo libero.

Il criterio di selezione: criterio, perche, rinvio

Dal luglio 2026 ogni curated dichiara **perché entra in archivio**, con un badge sopra la description (homepage, indice /curated/ e pagina singola) e una riga di framing rivolta al lettore (che spiega come usare i badge, non la politica editoriale interna).

I tre campi sono opzionali con fallback totale: un curated senza di essi rende esattamente come prima, nessun build si rompe. Uno slug scritto male non genera errori: il badge semplicemente non appare (il template valida contro src/_data/criteri.json, che è la fonte di verità per slug ed etichette visibili).

- criterio — **uno solo** (mai doppie assegnazioni), dalla lista chiusa:

Slug	Significato
lettura-di-mercato	mappa la struttura economica di un settore
referimento-tecnico	risorsa fondativa del tema
metodo-di-lavoro	cosa cambia nel processo o nell'organizzazione del lavoro

Slug	Significato
strumento-concettuale	tesi o categoria da portare su altri terreni
dato-che-corregge	evidenza (numero, trend o caso) che smentisce un consenso
elementi-di-scenario	aggiorna il quadro su un fronte che il sito segue — segnale, caso, storia esemplare. Ultima scelta, mai prima scelta

- perche — una sola frase tra virgolette doppie (senza doppie virgolette interne), mostrata sotto il badge. Non è un secondo riassunto: è la ragione d'archivio condensata, quasi sempre già presente nella description (“entra nel sito perché...”).
- rinvio — opzionale, path interno del pezzo con cui questo curated dialoga (es. /curated/2026-06-26-ottaviani-data-journalism-ai-reuters/). Reso come “In dialogo con →” nella pagina singola, con titolo risolto automaticamente. Si usa solo se il dialogo è esplicito nel commento.

Regole editoriali: nessuna casella “altro” — se nessun criterio si applica, il pezzo probabilmente non deve entrare (la lista è anche filtro in ingresso). Criterio e glossa finiscono nel JSON-LD della pagina singola (keywords e abstract), quindi sono estraibili dai motori generativi.

Se la lista dei criteri cambia, aggiornare due posti: `src/_data/criteri.json` (governa la resa) e la skill pubblica-articolo (governa la generazione). La legenda dei criteri sull'indice /curated/ (elemento <details> sotto la riga di framing) si genera automaticamente da `criteri.json`: non va toccata a mano.

concepts: lista di nomi dalla tassonomia concetti (sezione 5). Usa [] se nessuno si applica. **I nomi sono case-sensitive** e devono coincidere esattamente con un name in `conceptsIndex.js` — altrimenti vengono scartati (vedi sezione 5 per il controllo automatico).

I tags dei curated ora contano come Argomenti — ma solo se sono “riconosciuti”

I tag liberi che scrivi in tags: (oltre a curated) vengono **tradotti** in un Argomento vero — visibile in /indice/, /temi/ e con una pagina /tag/... propria — tramite la mappa in `src/_data/curatedTagAliases.js`. Funziona così:

- Se il tag che scrivi (case-insensitive) è già una chiave in `curatedTagAliases.js` → l'articolo conta per l'Argomento corrispondente. Esempio: scrivi ai minuscolo, l'articolo compare sotto l'Argomento AI.
- Se il tag **non** è in quella mappa → resta puramente decorativo: non genera nessuna pagina, non compare in nessun indice. Non è un errore, ma è facile non accorgersene.

Quando introduci un tema nuovo nei curated, controlla `curatedTagAliases.js`. Se il tag non c'è:

1. Decidi il nome canonico dell'Argomento (Title Case, in italiano per coerenza con gli altri — es. Sorveglianza, non surveillance).
2. Aggiungi la riga "tuo-tag-minuscolo": "Nome Canonico" in `curatedTagAliases.js`.
3. Aggiungi "Nome Canonico" all'array del cluster giusto in `src/_data/clusters.js` (sezione 7) — altrimenti l'Argomento esiste ma non ha un Tema e non appare in /temi/.
4. Facoltativo: aggiungi una descrizione in `tagDescriptions.json`.

Se invece il tema è in realtà un'entità (una persona, un paese, un'istituzione) e non un argomento trasversale, probabilmente appartiene ai Concetti (sezione 5), non agli Argomenti — è il caso tipico dei nomi di paese: vanno nel campo `concepts`, non in `tags`:

Dal settembre 2026 non devi più ricordartene da solo. `scripts/check-tags.js` gira nel prebuild insieme a `sync-concepts.js` e segnala in console tre cose, in ordine di urgenza: gli **alias mancanti** (il tag corrisponde a un Argomento che esiste già in `clusters.js` — perdita secca, si risolve con una riga), i **tag decorativi ricorrenti** sopra le tre occorrenze (promuoverli o lasciarli, purché sia una scelta), e gli **Argomenti orfani** (alias che puntano a un nome assente da `clusters.js`: la pagina /tag/ esiste ma non ha Tema né breadcrumb). I nomi di paese sono esclusi dal controllo di proposito. Come `sync-concepts.js`, avvisa e non blocca; `node scripts/check-tags.js --all` elenca anche la coda sotto soglia.

Body

Opzionale. Si usa solo per commenti molto estesi. Di solito il frontmatter basta.

3. Pubblicare una nota del Lab

Solo in inglese, per scelta

Il pubblico della sezione (revisori, potenziali endorser arXiv, target journal) legge solo in inglese, ed è la lingua in cui si cerca il cluster semantico su cui punta il GEO del progetto (*ontological taxonomy, inter-annotator agreement* ecc.). Il layout `lab-note.njk` imposta `lang: en` di default — non va specificato nel frontmatter delle singole note, né serve gestire coppie di file o link incrociati.

URL — `/lab/`, dall'agosto 2026

La sezione stava sotto `/episteme-advisory/lab/` ed è stata spostata in root: la ricerca annidata dentro la società for-profit indeboliva la credibilità della ricerca stessa. Episteme Advisory ora linka il Lab come ricerca indipendente, non come propria vetrina. I vecchi URL rispondono con un 301 definito in `netlify.toml` — **non rimuoverlo**, i link alle note sono già circolati.

L'URL è impostata via permalink, non dalla posizione dei file: `src/lab.njk` ha permalink: `"/lab/"` in frontmatter `src/lab/lab.11tydata.js` è un directory data file che assegna a ogni nota permalink: `"/lab/<nome-file-senza-estensione>/"`, data compresa nello slug (Eleventy la toglierebbe di default da `fileSlug/filePathStem` — per questo serve lo script invece di un semplice `.json`)

I file sorgente restano comunque in `src/lab/` — solo l'URL di output cambia.

La cornice del Lab è inglese

Il Lab ha `lang: en` e, dall'agosto 2026, anche la **cornice** inglese: navigazione, piede, briciole, skip link, barra di consenso. Lo governa `isENSurface` in `base.njk`, che considera superficie inglese tutto quello che sta sotto `/en/` **oppure** sotto `/lab/` (sezione 8). Lo switch di lingua su queste pagine dice IT e porta alla home italiana. Le briciole delle note sono `The programme · Lab`, dove «The programme» è `/en/`.

Conseguenza da tenere presente: la fascia Lab della home italiana continua a linkare `/lab/`, quindi un lettore italiano che clicca «Tutte le note» finisce in una cornice inglese. È voluto — le note sono in inglese comunque — ma se un giorno la cosa desse fastidio, il punto da cambiare è una riga sola in `base.njk`.

Niente retrodatazioni. Ogni nota va datata il giorno reale in cui viene pubblicata — non si simula una cronologia pregressa. Se serve raccontare il lavoro già fatto prima di aprire il log, si scrive una nota “stato dell’arte”/“project overview” datata oggi (vedi il primo esempio in `src/lab/`), non una serie di note con date fittizie nel passato.

File

Posizione: `src/lab/YYYY-MM-DD-slug.md`.

Frontmatter

```
---
layout: layouts/lab-note.njk
title: "Note title"
date: 2026-07-03
---
```

Campi opzionali

```
project: "Post-cognition" # a quale filone di ricerca appartiene — usalo solo se ce n'è più di uno
stage: "Writing & submission" # a che punto del processo si trova QUESTA nota — vedi tassonomia sotto
discipline: ["Epistemology", "Statistics"] # a quali campi del sapere attinge — non visibile, ...
```

```
method: ["Quantitative analysis"] # con che tipo di evidenza/tecnica lavora – non vis  
description: "1-2 sentences" # solo se serve per meta/OG, non obbligatoria
```

Niente campo tags. Le note del Lab non hanno etichette libere in testa: con un solo filone di ricerca alla volta sarebbero quasi sempre identiche da una nota all'altra, e non essendo linkate a /tag/slug/ non aggiungerebbero navigazione reale — solo rumore.

Collegamento ai concetti del sito — tagging manuale e selettivo

Le note del Lab **non vengono processate da sync-concepts.js** (che legge solo curated/ e learning/). Questa è una scelta deliberata: ogni nota del lab condivide un nucleo di concetti (epistemia, post-cognition, AI) che se aggiunti automaticamente inflazionerebbero quelle pagine concetto senza aggiungere segnale utile.

Il tagging del lab è manuale e selettivo: si aggiunge un articolo a un concetto in `conceptsIndex.js` solo quando la nota arricchisce concretamente quella voce — non per default. Esempio: una nota specificamente dedicata al ruolo degli atti illocutori nel Pre-Step 0 vale come link nella pagina “Austin, John L.” e “atti illocutori”; una nota operativa sullo stato del corpus non arricchisce nessun concetto.

Formato da usare per un articolo lab in `conceptsIndex.js`:

```
{ title: "Titolo della nota", url: "/lab/slug-nota/", _source: "lab" }
```

Nell'indice gli articoli con `_source: "lab"` appaiono con il badge LAB in verde Episteme (#0F6E56).

I concetti fondativi del progetto (post-cognition, epistemia, tassonomia D1–D7, atti illocutori) puntano alla landing /lab/ come punto di accesso generale — non alle singole note.

Tassonomia del Lab — separata da quella del sito

Tre assi indipendenti, pensati apposta per non mescolarsi con `clusters.js/conceptsIndex.js/Indice` (che si riempirebbero di “Epistemologia” ripetuta su ogni nota). Nessuna registrazione centrale, nessun controllo automatico — sono liste aperte, aggiungi un valore quando serve:

- **discipline** (lista) — campo/i del sapere. Partenza: *Epistemology, Philosophy of language, Computational linguistics / NLP, Statistics, Cognitive science, AI governance & regulation*.
- **stage** (valore singolo) — fase del processo in cui si trova la nota. Partenza: *Framework design, Taxonomy development, Corpus construction, Empirical validation, Data analysis, Writing & submission, Dissemination*.
- **method** (lista) — tipo di evidenza/tecnica. Partenza: *Qualitative coding, Quantitative analysis, Statistical modeling, Model evaluation / benchmarking, Literature review, Annotation / inter-annotator agreement*.

Solo stage è visibile (badge in testata, nella card dell'indice, nella striscia home). discipline e method restano metadati strutturati nel JSON-LD (about/keywords) — segnale per la ricerca semantica/GEO senza aggiungere pillole in pagina.

Cosa NON fa questo layout (di proposito)

Niente abstract obbligatorio, niente serie, niente immagine hero, niente TOC, niente share bar, niente post correlati, niente tag liberi. `project/stage` sono etichette visive che non toccano `clusters.js` né `conceptsIndex.js`. Le date si formattano con il filtro `formatDateEN` (inglese), non `formatDate` (italiano, usato dal resto del sito). Il corpo della nota (`.article-full--lab .article-body`) è a 16px invece dei 18px dei writings — restano note di cantiere, non saggi.

Visibilità

Il Lab è raggiungibile da quattro punti, tutti automatici:

- **voce «Lab» nella navigazione inglese** (non in quella italiana, che resta a sei voci)
- **fascia Lab nella home italiana** (`.home-band--lab` in `src/index.njk`): ultima nota con anteprima vera, più le successive
- **fascia Lab nella home inglese** (`src/en/index.njk`), dove pesa di più — è da lì che viene il verde della pagina

- un rimando da `src/episteme-advisory.njk`, che linka il Lab come ricerca indipendente

Tutte leggono `collections.lab`: si aggiornano da sole quando esce una nota, non richiedono manutenzione.

Documenti citati — link automatico e Learning Log

Le note del Lab citano spesso un documento di lavoro (tassonomia, coding manual, corpus, script, bibliografia). Questi documenti sono anche scaricabili dal Learning Log (`/learning/`), in una sezione verde separata “Dal Lab” — colore Episteme Advisory (`#0F6E56`), diverso apposta dal blu delle risorse standard del Learning Log, per segnalare che sono materiali di lavoro grezzi, non risorse finite.

Il meccanismo è tutto nel frontmatter — non si tocca `learning.njk` né si crea alcun file in `src/learning/`.

0. Il nome del file. I documenti del programma di ricerca usano uno schema leggibile una volta salvati sul desktop di un lettore: `cammarano-<argomento>-<versione>.pdf` (per esempio `cammarano-post-cognitive-validation-v6.pdf`). I nomi vecchi in stile `11_2026_academic_v6.pdf` sono stati rinominati e i vecchi percorsi hanno un 301 in `netlify.toml`. **Se rinomini un documento già pubblicato:** aggiorna il file: nel frontmatter e le chiamate `{% labdoc %}` di tutte le note che lo citano, aggiungi il redirect, e ricontrolla `src/en/index.njk`, che linka tre PDF direttamente.

1. Il file fisico va in `src/downloads/` (stessa cartella piatta delle risorse esistenti, copiata in automatico da `addPassthroughCopy` in `.eleventy.js`). Formato preferito: **PDF**, anche per documenti nati come testo — evita ambiguità di formato ed è leggibile ovunque senza Word. Script (`.py`) e dati strutturati (`.bib`, `.json`) restano nel loro formato nativo. Se un file contiene path assoluti della propria macchina (es. `/Users/nome/Documents/...`), toglierli prima di caricarlo — sono superflui per chi scarica e rivelano struttura di cartelle privata.

2. Nel frontmatter della nota che introduce il documento, aggiungere un blocco `documents::`

```
documents:
  - file: "nome_file.pdf"
    label: "Titolo leggibile del documento"
    version: "2.0"           # facoltativo — numero versione, o testo libero ("v4", "Addendum a v1.0")
    id: "id-stabile"        # facoltativo — solo se questo documento SOSTITUISCE una versione precedente
```

Questo blocco va **solo sulla nota che introduce il documento** (di norma quella dedicata, non le note successive che lo ricitano di sfuggita). Genera in automatico: il link nel testo (via shortcode, punto 3), la card verde nel Learning Log con data (quella della nota), versione e link “← Torna alla nota”.

3. Nel corpo della nota — di QUALSIASI nota, non solo quella che introduce il documento — sostituire la menzione del file con lo shortcode:

```
Document: {% labdoc "nome_file.pdf" %}
```

Lo shortcode cerca il filename in tutti i `documents:` dichiarati nel Lab e genera il link di download. Se il file non è ancora registrato in nessun frontmatter, torna al vecchio stile testuale (“`nome_file.pdf`”) invece di rompere la build — un promemoria visivo che manca il passo 2.

4. Versioni che sostituiscono un documento precedente. Se una nota pubblica una nuova versione di un documento già registrato (es. una Addendum viene poi assorbita in un Coding Manual v2.0), dare ai due blocchi `documents:` lo **stesso id**. Il Learning Log mostra automaticamente solo la versione più recente per ogni id (per data della nota); quella precedente sparisce dalla griglia “Dal Lab” ma resta scaricabile dal suo URL e dal link `{% labdoc %}` nella nota vecchia — nulla si cancella, cambia solo cosa compare in vetrina. Senza id, ogni documento ha la propria card indipendente (il caso normale).

La pagina dei deliverables. Gli stessi `documents:` alimentano anche `/en/deliverables/`, che li raggruppa per funzione — paper, framework, corpus, codice, bibliografia — e marca come superseded ogni versione che ne ha una più recente con lo stesso id. Le sezioni e l’assegnazione stanno in `src/_data/deliverableGroups.js`. **Un documento nuovo non richiede di toccare quel file:** se non è assegnato per id né per nome, cade nel gruppo della sua estensione. Si interviene solo quando la rete di sicurezza lo mette nel posto sbagliato.

Licenza: tutte le card “Dal Lab” mostrano CC BY-NC-SA 4.0, la stessa del resto del Learning Log — non è un campo per documento, è fissa nel template.

Tre ricette pratiche:

- *Nuova nota che cita un documento mai visto prima* → aggiungere `documents`: con i suoi dati, mettere il file in `src/downloads/`, usare `{% labdoc %}` nel testo.
 - *Nuova nota che ricita un documento già registrato* → nessun `documents`: da aggiungere, solo `{% labdoc "nome_file.pdf" %}` nel testo (il link si risolve da solo).
 - *Nuova nota che pubblica una versione aggiornata di un documento esistente* → `documents`: con lo stesso id del documento precedente; il file vecchio non si tocca né si cancella.
-

4. Aggiungere una serie

1. I saggi di una serie usano il campo `series`: "Nome serie, I" (con numero romano).
2. Aggiungere la descrizione della serie in `src/_data/seriesDescriptions.json`:

```
{  
  "Nome serie": "Descrizione della serie per la pagina /serie/."  
}
```

3. Nessun altro intervento: le pagine `/serie/` e `/serie/nome-serie/` si generano automaticamente.

Attenzione al passo 2. Il Sommario ragionato in `home` itera `seriesDescriptions.json`: una serie senza descrizione non compare lì. Non è un errore e non lo vedi nel log — è un’omissione silenziosa, l’unica del sistema.

5. Tassonomia concetti (`_data/conceptsIndex.js`)

Questo file è la **fonte di verità** per l’indice analitico, le pagine `/concetti/slug/` e il grafo. È il livello più specifico dei tre (Temi → Argomenti → **Concetti**): persone, teorie, testi, istituzioni, luoghi e paesi citati con peso argomentativo.

Struttura di una voce

```
{  
  name: "Axelrod, Robert",      // nome esatto, usato come ID  
  type: "persona",              // persona | teoria | testo | istituzione | luogo | paese  
  articles: [                   // articoli writings che lo citano  
    { title: "L'ombra del futuro", url: "/writings/2026-04-15-lombra-del-futuro/" }  
  ],  
  note: "Testo facoltativo.",    // nota descrittiva (opzionale)  
  geo: { modo: "diretta", paesi: ["Stati Uniti"] }, // OBBLIGATORIO – vedi «La geografia»  
  sameAs: [                     // agganci all'entità reale (opzionale)  
    "https://www.wikidata.org/wiki/Q583438",  
    "https://en.wikipedia.org/wiki/Robert_Axelrod_(political_scientist)"  
  ],  
  related: [                     // legami dichiarati verso altri concetti (opzionale)  
    { name: "The Evolution of Cooperation", why: "Axelrod mette in gara le strategie..." }  
  ]  
}
```

Aggiungere un nuovo concetto

1. Aprire `src/_data/conceptsIndex.js`
2. Aggiungere la voce nell’array, rispettando il formato sopra
3. Per i **writings**: inserire subito gli articoli che lo citano nell’array `articles`

4. Per i **curated** e **learning**: lasciare `articles: []` e aggiungere il nome nel campo `concepts:` del file — il sistema li unisce automaticamente a `build time` tramite `sync-concepts.js` (`prebuild`) e `mergedConceptsIndex` (`build`)
5. Per le **note del lab**: tagging manuale selettivo — vedi sezione 3
6. Fare il `build`: la pagina `/concetti/nome-slug/` appare automaticamente

Tipi validi — attenzione a paese vs luogo

Tipo	Esempi
persona	Axelrod, Robert · Trump, Donald · Arendt, Hannah
teoria	scenario planning · dual use · allineamento AI
testo	The Evolution of Cooperation · Antifragile
istituzione	Anthropic · Palantir · DARPA · Studio Ghibli
luogo	Bologna · Libano / Beirut · Taiwan / TSMC
paese	Europa · Stati Uniti · Cina · Russia · Giappone

Ogni nome di stato/nazione è paese, non luogo — anche quando nel linguaggio comune lo chiameremmo “un luogo” (es. Giappone, Germania, Iran). `luogo` è riservato a città, regioni, snodi geografici specifici che non sono uno stato (Bologna, Beirut, Taiwan come territorio conteso). I due tipi finiscono in sezioni diverse dell'indice analitico (“Paesi” e “Luoghi”): se un nome non si trova dove lo cerchi, controlla prima l'altra sezione.

Aggancio all'entità reale — il campo `sameAs`

Array di URL che identificano l'entità di cui la voce parla: il Q-id di Wikidata e, dove esiste, la voce Wikipedia (italiana quando c'è, inglese altrimenti). È facoltativo e va lasciato vuoto quando l'aggancio non è certo: **un Q-id sbagliato è peggio di nessun Q-id**, perché afferma in forma leggibile dalle macchine un'identità falsa.

Al 25 settembre 2026: **216 voci su 268 agganciate a Wikidata**. Le scelte della revisione (Q-id accettati, scartati, termini di conio) sono registrate in `scripts/wikidata/decisioni.json`, che sostituisce il vecchio `wikidata-da-rivedere.md`. **I termini di conio non hanno Q-id**: restano ancorati solo in `/ns/`. Per cercare candidati si usa `scripts/wikidata/fetch.py` e `search2.py`, da lanciare dal Mac (serve la rete; usano `curl` per i certificati).

Il campo finisce nel JSON-LD della pagina `/concetti/slug/`, costruito dal filtro `conceptJsonLd` in `.eleventy.js`. Ogni pagina concetto si dichiara `DefinedTerm` di un `DefinedTermSet` (`/indice/#set`); l'identità dell'entità reale sta nel nodo `about` (`Person`, `Organization`, `Place`, `Country`, `CreativeWork` secondo il tipo), mentre per i concetti astratti il `sameAs` sta sul termine stesso. La nota integrale entra in `description`.

La geografia — il campo `geo` (obbligatorio da settembre 2026)

Alimenta la carta «Dove guarda questo sito» in `home`. Forma: `geo: { modo: "...", paesi: [...] }`.

modo	Quando	Peso sulla carta	Esempi
diretta	Entità concrete: persone (cittadinanza, tutte se più d'una), istituzioni (sede), testi (origine), luoghi e paesi (sé stessi); concetti che parlano di un luogo preciso	1 se il luogo è citato nel pezzo, 0,5 se arriva tramite un concetto	Diegoli, Gianluca → Italia; Stratechery → Stati Uniti, Taiwan; dottrina Gerasimov → Russia

modo	Quando	Peso sulla carta	Esempi
teorico	Concetti astratti che portano il luogo di chi li ha formulati	0,25	antifragilità → Libano, Stati Uniti (Taleb); epistemia → Italia (Quattrociocchi)
nessuna	Concetti trasversali senza teorico; tutti i termini di conio (non ereditano l'Italia dall'autore)	non contano	editoria, allineamento AI, post-cognition

Regole:

- **Nomi dei paesi:** esattamente quelli di `src/_data/geoPaesi.json` (in italiano, con codice ISO). Un paese nuovo si aggiunge lì, sotto stati.
- **Forme-stato:** l'UE (e l'Europa come progetto politico) si scrive "UE" e colora tutti i membri; l'Italia ha voce propria (menzioni Italia + UE). Le sole regioni ammesse sono in regioni (UE, Africa subsahariana) e distribuiscono il peso pieno su ogni Stato membro.
- **Stati storici → Stato attuale erede** (Prussia, Impero tedesco, DDR → Germania; Regno di Napoli → Italia; Atene → Grecia). Per l'URSS la repubblica effettiva, mai l'URSS.

Il controllo `scripts/validate-geo.js` gira in `prestart` e `prebuild` e **blocca** se una voce non ha geo, se il modo non è valido, se nessuna ha paesi o gli altri modi no, o se un paese non è in `geoPaesi.json`. Per rigenerare il campo su tutto l'indice dalle decisioni registrate: `python3 scripts/wikidata/apply_geo.py`.

Legami dichiarati fra concetti — il campo `related`

Archi concetto↔concetto asseriti a mano, distinti dalla co-occorrenza che il sito calcola da solo (i chip «Concetti vicini», dal filtro `relatedConcepts`). Un arco esiste perché qualcuno lo ha scritto, e porta con sé la ragione.

Tre regole, fatte rispettare dal build:

- **Si dichiara una volta sola.** Il filtro `assertedLinks` lo rende su entrambe le pagine. Dichiararlo due volte fa fallire il build.
- **Il why si scrive come relazione, non come direzione.** «coautori di prospect theory» si legge bene da tutte e due le parti, «ha influenzato X» no. Massimo 160 caratteri.
- **Massimo 5 legami per voce.** Un elenco più lungo torna a essere rumore.

Il criterio per decidere se un legame esiste: *le due note si nominano a vicenda?* Se la nota di A parla di B — o della cosa che B è — il legame è già argomentato e il why deve solo renderlo esplicito. Se condividono soltanto un articolo o un po' di lessico, è scenografia.

A settembre 2026: 52 legami su 44 voci dichiaranti. I legami **non entrano ancora nel grafo**: `/mappa/` e il riquadro HEB continuano a disegnare la sola co-occorrenza.

Un effetto collaterale da conoscere: `related` duplica i nomi nel file. Dichiarare un legame significa scrivere il nome di una voce dentro un'altra voce — e quel nome può trovarsi molto prima della voce vera e propria. `sync-concepts.js` cercava il concetto alla prima occorrenza del nome nel file: con i legami attivi, quella prima occorrenza può essere un `related` altrui, e l'articolo finiva appeso alla voce sbagliata. Peggio: il controllo anti-duplicazione guardava la voce giusta, quindi il problema si ripeteva a ogni build, in silenzio, accumulando copie.

Corretto a settembre 2026 — ora la voce si cerca dalla sua **definizione** (`name: seguito da type:`), non dalla prima occorrenza del nome. La regola per chi metterà mano a questi script: *il nome di un concetto non è una chiave univoca nel file, lo è solo la sua definizione*. Se un giorno si aggiungessero altri campi che citano concetti per nome, vale lo stesso avvertimento.

Il controllo automatico — niente più concetti persi in silenzio

Per anni il sistema ha scartato **senza nessun avviso** qualsiasi nome in `concepts`: di un `curated` che non corrispondesse a una voce già presente in `conceptsIndex.js`. Risultato: articoli pubblicati con concetti che restavano invisibili nell'indice e nel grafo, senza che nessuno se ne accorgesse — a volte per mesi.

Ora il build segnala il problema in console. Ogni volta che esegui `npm run build` o `npm start`, se un `curated` o un `learning` cita un concetto non registrato compare un blocco come questo:

```
△ CONCETTI NON REGISTRATI (ignorati nell'indice/grafico) – aggiungili a src/_data/conceptsIndex.js:
./src/curated/2026-06-21-esempio.md -> "Nome Concetto"
./src/learning/2026-03-13-jaynes-guida-lettura.md -> "Jaynes, Edwin T."
```

Se lo vedi: apri `conceptsIndex.js`, aggiungi la voce mancante (sezione precedente), rifai il build. L'avviso scompare quando tutto è registrato. **Su Netlify lo stesso messaggio compare nei log di deploy** (Deploy → log) — quindi anche pubblicando solo via `git push`, senza build locale, il problema resta visibile.

Il controllo che blocca — `validate-concepts.js`

Gira nel `prebuild` e nel `prestart`, dopo `sync-concepts.js`, e **fa fallire il build** invece di limitarsi ad avvisare. Controlla:

- `related` — il nome puntato esiste, non è la voce stessa, non è ripetuto, l'arco non è dichiarato in entrambe le direzioni, il `why` c'è e sta sotto i 160 caratteri, e la voce non supera i 5 legami;
- `articles` — ogni url ha un file sorgente corrispondente (gli url di sezione come `/lab/` sono esclusi: sono pagine indice);
- `sameAs` — è un array di URL `http(s)`.

Subito dopo gira `validate-geo.js`, che controlla il campo `geo` (vedi «La geografia»).

Un refuso in un nome produceva prima un legame che spariva in silenzio. Ora il build si ferma e dice quale voce.

L'indice come dataset pubblico

Da settembre 2026 lo stesso indice esce anche in forma aggregata, generata a build time dalla stessa fonte di verità: nessuno di questi file si mantiene a mano.

URL	Che cos'è
<code>/concetti.json</code>	L'indice come JSON-LD, filtro <code>conceptSchemeJsonLd</code> . Radice insieme <code>skos:ConceptScheme</code> , <code>DefinedTermSet</code> e <code>Dataset</code> , con licenza CC BY 4.0, versione, <code>creator</code> con ORCID e un blocco <code>cc:statistics</code> . Ogni voce porta <code>prefLabel</code> , la nota integrale in <code>skos:note</code> , <code>skos:exactMatch</code> verso Wikidata e Wikipedia, l'entità tipizzata in <code>about</code> , gli articoli in <code>subjectOf</code> e i legami.
<code>/llms.txt</code>	Indice conciso: <code>writings</code> con <code>description</code> , le 261 voci una riga ciascuna, i 116 <code>curated</code> con il loro perché, i puntatori ai file espansi. ~85 KB.
<code>/llms-full.txt</code>	Gli stessi contenuti con note e descrizioni integrali. ~430 KB.
<code>/ns/</code>	I tre termini propri usati nel dataset, dichiarati e dereferenzabili.

Perché esiste `/ns/`. SKOS e `schema.org` coprono quasi tutto, ma non il `why` di un legame: `skos:related` è una relazione nuda. Il legame è quindi reificato in un nodo `cc:Link` con `cc:target` e `cc:why`, e gli stessi legami sono ripetuti in forma nuda in `skos:related` per chi conosce solo SKOS. Insieme a `cc:type` sono i tre termini dichiarati nel namespace `https://claudiocammarano.com/ns#`. Un termine proprio non dichiarato è un termine inventato: la pagina è la dichiarazione.

Deposito e DOI. In dataset/ ci sono README.md (schema, metodo, provenienza, limiti dichiarati), CITATION.cff e zenodo-metadata.md con i campi già compilati. Il caricamento su Zenodo va fatto a mano dal proprio account. Depositato il 19 settembre 2026. I DOI sono due e non sono intercambiabili: **10.5281/zenodo.22843731** è quello concettuale — non cambia mai, risolve sempre all'ultima versione, ed è quello che va citato e messo nelle pagine; **10.5281/zenodo.22843732** identifica la versione 1.0.0 e solo quella. Stanno entrambi in src/_data/dataset.js (doi e versionDoi) e finiscono entrambi in identifier dentro /concetti.json. Per le versioni successive: alzare version e released, scrivere il changelog, usare «New version» sullo stesso record Zenodo e aggiornare versionDoi con il DOI nuovo — doi non si tocca.

6. Come funziona la navigazione semantica

Il sito ha tre livelli di navigazione, dal più ampio al più specifico:

TEMI (5 cluster) /temi/
↓ ogni tema raggruppa più Argomenti
ARGOMENTI (~49 tag) /tag/slug/
↓ ogni Argomento elenca gli articoli (writings + curated) che lo usano
CONCETTI (persone, teorie...) /concetti/slug/
↓ entità citate con peso argomentativo dentro gli articoli

Più l'**indice analitico** (/indice/), che mostra tutti e tre i livelli insieme in un'unica pagina — pensato apposta perché un visitatore capisca subito su quale livello si trova.

Temi (/temi/) e Argomenti (/tag/slug/)

- I **Temi** sono i cluster definiti in clusters.js (sezione 7): 5 raggruppamenti larghi.
- Gli **Argomenti** sono i singoli tag — quelli storici dei writings (category:) e quelli più recenti emersi dai curated (tags:, tradotti via curatedTagAliases.js, sezione 2).
- Ogni Argomento ha una pagina /tag/slug/ che mostra **sia** i writings che lo usano in category: **sia** i curated il cui tag si traduce in quell'Argomento — stessa pagina, card diverse per tipo.
- Le descrizioni (opzionali) vivono in tagDescriptions.json (per gli Argomenti) e temiDescriptions.json (per i Temi).

Concetti (/concetti/slug/)

Ogni concetto ha una pagina propria con: i “concetti vicini” (chip cliccabili — concetti che condividono almeno un articolo), la lista degli articoli collegati, e una **finestra sul grafo globale** — una porzione ritagliata del grafo completo, centrata sul concetto, con una minimappa che mostra dove si trova rispetto all'insieme. Link “Vedi mappa completa →” per aprire /mappa/ centrata sullo stesso nodo.

Il grafo è **uno solo**: le coordinate dei nodi sono calcolate una volta a build time da graphLayout.js (simulazione a forze deterministica, nessuna libreria esterna) ed esposte come JSON statico su /graph-data.json; il disegno è *hierarchical edge bundling*, in src/js/heb.js, che ha sostituito nell'agosto 2026 il vecchio grafo a forze visibile a schermo. Ogni pagina concetto e la pagina /mappa/ lo leggono e ne mostrano porzioni diverse — non sono grafi separati.

Il grafo è scope-limitato a Concetti e articoli. Temi e Argomenti non ci entrano: è una scelta deliberata, per non sovraccaricare una visualizzazione già densa.

Visibile solo da schermi ≥ 900px. Su mobile, sia le pagine concetto sia /mappa/ mostrano la sola lista testuale — nessuna modifica al mobile rispetto a prima dell'introduzione del grafo a finestra.

Il flusso dei dati, in breve

- **Writings:** Argomento ← category: nel frontmatter. Concetto ← voce manuale in conceptsIndex.js (sezione 1).
- **Curated:** Argomento ← tags: nel frontmatter, tradotto via curatedTagAliases.js (sezione 2). Concetto ← concepts: nel frontmatter, automaticamente sincronizzato via sync-concepts.js (sezione 5).

- **Learning:** Concetto ← concepts: nel frontmatter, automaticamente sincronizzato via sync-concepts.js (sezione 5). Nell'indice gli articoli learning appaiono con il badge lr in ambra.
- **Lab:** Concetto ← tagging **manuale e selettivo** direttamente in conceptsIndex.js, con URL /lab/slug/ (vedi sezione 3). sync-concepts.js non legge src/lab/ — ogni nota del lab viene collegata solo ai concetti che arricchisce concretamente, non per default.

Non esiste un comando per “aggiornare l'indice semantico”. È lo stesso identico git push (o npm run build) con cui pubblichi il sito. Se i dati a monte sono corretti, il build successivo rigenera tutto: indici, pagine concetto, grafo, Temi, Argomenti — niente da “richiamare” a parte.

Il Sommario ragionato (home)

Pannello tipografico **in coda alla home** italiana, ultima delle fasce orizzontali: l'albero della tassonomia sul modello del *Système figuré* dell'Encyclopédie (da cui il nome interno dei file, sistema-figurato). Mostra i 5 Temi come rami, con una selezione di Argomenti, concetti-campione in corsivo, le serie e i rimandi a /mappa/ e /indice/.

- **Template:** src/_includes/components/sistema-figurato.njk, incluso da index.njk.
- **Si aggiorna da solo a ogni build:** contatori (temi/argomenti/concetti/serie), elenco argomenti per ramo, serie. Legge clusters.js, la collection mergedConceptsIndex e seriesDescriptions.json.
- **Unico dato curato a mano:** i concetti-campione (*specimen*) in src/_data/sistemaFigurato.js — le chiavi devono combaciare alla lettera con quelle di clusters.js. Lì si regola anche maxArgomenti (quanti argomenti per ramo prima dell'ellissi) e il colorKey che aggancia il pallino di ogni ramo ai colori del grafo (--graph-cluster-*).
- **CSS:** sezione “Sistema figurato” in fondo a style.css (classi sf-*).
- I link degli specimen puntano a /concetti/slug/: se rimuovi un concetto da conceptsIndex.js, controlla che non sia citato come specimen.

7. Temi e Argomenti (_data/clusters.js)

clusters.js è la tassonomia dei **Temi**: un oggetto che mappa ogni nome di Tema a un array di Argomenti.

```
module.exports = {
  "Epistemologia & AI":      ["Filosofia", "Epistemologia", "AI", "Scrittura", "Vibe Coding", "Clau..."],
  "Geopolitica & potere":    ["Geopolitica", "Teoria dei giochi", "Difesa", "Europa", ...],
  "Editoria & comunicazione": ["Comunicazione", "Dual Use", "Media", "Editoria", ...],
  "Italia & istituzioni":     ["Italia", "Bologna", "Humanities", "Femminismo", ...],
  "Economia & lavoro":       ["Economia", "Lavoro"]
};
```

Cinque Temi in tutto (il quinto, “Economia & lavoro”, è nato dai curated: nessuno dei 4 Temi originari aveva una casa naturale per quei contenuti).

Per aggiungere un Argomento a un Tema: inserirlo nell'array del Tema corrispondente. /temi/, /indice/ e il breadcrumb si aggiornano al prossimo build.

Per i writings: basta scrivere il nome esatto in category: (sezione 1).

Per i curated: il tag in tags: deve prima essere tradotto in curatedTagAliases.js (sezione 2) — è un passaggio in più, perché i tag dei curated sono scritti liberi e minuscoli, non nel formato canonico.

Le descrizioni dei Temi vivono in temiDescriptions.json, quelle degli Argomenti in tagDescriptions.json — entrambe facoltative, entrambe scritte in prima persona e ancorate ai contenuti reali del sito, non enciclopediche.

8. La superficie inglese (/en/)

Il sito è uno, in due lingue, non due marchi: la testata blu e l'identità visiva restano le stesse, cambiano navigazione e lingua dei contenuti.

Struttura e regola non negoziabile

/ è italiano, /en/ è inglese, URL statici. **Nessun redirect per lingua o per geografia**, mai: niente su IP, niente su Accept-Language. Una richiesta a / da un IP statunitense deve restituire la home italiana, 200, senza catene di redirect. È una scelta di posizionamento, non una dimenticanza: il posizionamento organico italiano va protetto.

Che cosa esiste in inglese

/en/ (home del programma di ricerca, cinque fasce), /en/start-here/, /en/about/, /en/episteme-advisory/, /en/colophon/ (versione breve), /en/visualizations/measuring-bench/, più l'intero /lab/. Fuori perimetro per decisione: indice e mappa semantica in inglese, traduzione dell'archivio dei saggi.

Dichiarare una coppia di pagine equivalenti

Nel frontmatter di **entrambe**:

```
translation:
  lang: en          # o it
  url: /en/...
```

Genera hreflang reciproci, x-default sull'italiano, e fa puntare il bottone di testata alla pagina gemella. Senza translation non esce nessun x-default e il bottone porta alla home dell'altra opera — il comportamento corretto quando la gemella non esiste.

Le due home non sono gemelle (decisione di settembre 2026): / e /en/ sono due opere diverse, non la traduzione l'una dell'altra, e non hanno translation. Il bottone in testata non dice più IT/EN: dice «**Research EN**» sul lato italiano e «**Saggi IT**» su quello inglese, con un tooltip che spiega l'altra opera. Pagine gemelle vere oggi: Episteme Advisory (/episteme-advisory/ & /en/episteme-advisory/) e le poche altre dichiarate.

Non dichiarare translation fra pagine che non sono equivalenti. /da-qui/ e /en/start-here/ hanno funzioni diverse — uno guida chi esplora un archivio, l'altro chi sta decidendo se l'autore è un interlocutore — e dichiararle gemelle sarebbe un hreflang falso.

Le due variabili del layout, da non confondere

In base.njk:

- **isEN** — vero quando lang: en. Governa **solo i metadati**: <html lang>, og:locale.
- **isENSurface** — vero quando l'URL sta sotto /en/ **oppure** sotto /lab/. Governa **la cornice**: navigazione, piede, skip link, barra di consenso, etichetta e destinazione dello switch.

Sono due cose diverse perché il Lab è scritto in inglese ma vive fuori da /en/.

Regole di contenuto sulle pagine inglesi

- **Nessuna porta commerciale.** Irene Media non compare mai; Episteme Advisory una volta sola, in coda alla home, come contesto istituzionale della ricerca. Il piede inglese è un ramo separato apposta per questo.
- **Nessun link a contenuto italiano senza segnalarlo:** lang="it" hreflang="it" sull'ancora più un'etichetta visibile, «(in Italian)» o «(IT)».
- I due progetti del sito — il banco di misura e il paper — sono distinti e vanno tenuti distinti. Il raccordo che li mette in relazione è scritto in due punti, la home inglese e la pagina del banco, **con le stesse parole**: se ne cambi uno, cambia anche l'altro.

Aggiungere una pagina inglese

1. File in src/en/, frontmatter con permalink: /en/.../, lang: en, og_card, ed eventualmente translation.
2. Aggiungere l'URL a mano nella sezione statica di src/sitemap.njk.
3. Se deve stare in navigazione, aggiungerla al ramo inglese di header.njk.

Card social

og_card per l'immagine 1200×630, og_title quando il titolo della scheda deve dire più del <title>. Le card sono immagini fatte a mano in src/images/og/: non esiste un generatore in repo. La forma di casa è fissa — fondo #f5f5f0, filetto in testa (blu #1C0E80 per l'italiano, verde #0F6E56 per la superficie inglese), occhiello spaziato, titolo serif, dominio in basso.

9. Gli strumenti interattivi (il banco di misura)

Dove vivono

src/pages/banco-di-misura.html → /visualizations/banco-di-misura/ src/pages/measuring-bench.html → /en/visualizations/measuring-bench/

Non sono file statici copiati: sono template Eleventy con layout: layouts/base.njk e permalink esplicito. Stanno in src/pages/ proprio perché l'URL non corrisponde alla posizione del file. Fino ad agosto erano isole senza cornice, copiate da addPassthroughCopy; oggi prendono testata, navigazione e piede del sito nella lingua della pagina.

Perché il CSS è tutto sotto .bm

Lo strumento porta con sé un reset (*{margin:0;padding:0}) e regole su footer e h1. Globali, quelle regole azzererebbero i margini della testata e riscriverebbero il piede del sito. Sono quindi ristrette a .bm, il contenitore che avvolge lo strumento. **Se aggiungi una regola, tienila dentro .bm.**

Le variabili di colore dello strumento (--bg, --text, --blue, --green...) stanno su :root con nomi propri e non collidono con quelle del sito, che sono tutte --color-*. Il canvas le legge da document.documentElement con la funzione css(): **non spostarle sotto .bm**, il disegno del piano si spegnerebbe.

body.bench .container allarga il contenitore del sito da 800 a 1000px, perché lo strumento ne vuole 940.

Il modo incorporato

Il saggio *La forza della scrittura* mostra lo strumento dentro un iframe. Una riga in base.njk mette la classe embedded su <html> quando la pagina non è al primo livello: la cornice si nasconde e Google Analytics non conta una seconda visualizzazione. Se tocchi quel punto, controlla il saggio prima di pubblicare.

Debito noto — leggere prima di metterci mano

I due file condividono circa il **novanta per cento** del codice: stessa matematica, stesso canvas, stessa simulazione. Una correzione va applicata **due volte**, e nulla te lo ricorda. Se un giorno vale la pena rifattorizzare, la cosa giusta è un file solo con le stringhe separate per lingua.

Nel frattempo, le tre cose che divergono e che è facile dimenticare:

1. **Preset e verdetti sono tradotti, non paralleli.** Se cambi un caso in italiano, cambialo anche in inglese.
2. **I numeri:** l'italiano usa la virgola decimale, l'inglese il punto. La differenza sta nella funzione f2().
3. **L'ordine argomentativo è diverso di proposito.** L'italiano si apre dichiarandosi strumento di lettura del saggio; l'inglese apre con che cosa misura, perché il problema esiste e l'invito a muovere i cursori, e solo dopo rimanda al resto. Non allinearli.

10. Cosa NON toccare senza capire

File	Perché è delicato
.eleventy.js	Filtri e collections: un errore blocca il build
src/_includes/layouts/article.njk	Template di tutti i saggi

File	Perché è delicato
src/_includes/layouts/base.njk	Template base di tutto il sito. Contiene isEN/isENSurface (sezione 8), il rilevamento dell'iframe e il consenso GA
src/_includes/components/header.njk e footer.njk	Hanno due rami, italiano e inglese: modificare il ramo sbagliato non dà errore, cambia solo l'altra lingua
src/pages/banco-di-misura.html e measuring-bench.html	Gli stessi 850 righe di codice due volte, in due lingue (sezione 9)
netlify.toml	Redirect 301: Lab in root, vecchi permalink Medium, nomi dei PDF. Rimuoverne uno rompe link già circolati
src/sitemap.njk	La sezione delle pagine statiche è scritta a mano: una pagina nuova non entra da sola
src/css/style.css	Unico file CSS, ~7300 righe, tutto a selettori globali: una regola cambiata può toccare cinque pagine
src/_data/conceptsIndex.js	Fonte di verità per indice e concetti — 240 voci scritte a mano
src/_data/aiNotation.js	Fonte unica della notazione AI: un codice tolto qui fa fallire il build su tutti i pezzi che lo usano
src/_data/clusters.js	Tassonomia Temi/Argomenti — usata da /temi/, /indice/, breadcrumb e dal layout del grafo
src/_data/curatedTagAliases.js	Traduce i tag liberi dei curated in Argomenti — un nome cambiato qui “spegne” silenziosamente quell'Argomento per i curated già pubblicati
src/_data/graphLayout.js	Calcola le posizioni del grafo. Non richiede modifiche manuali: si rigenera da solo leggendo conceptsIndex.js e clusters.js
src/_data/sistemaFigurato.js	Specimen del Sommario ragionato in home — le chiavi devono combaciare con clusters.js, i nomi con conceptsIndex.js

11. Troubleshooting rapido

Un concetto/articolo che ho appena pubblicato non appare da nessuna parte → **Prima cosa da controllare:** lancia `npm run build` e leggi la console. Se il concetto è citato in un `concepts`: di un `curated` ma non registrato, ora compare un avviso `CONCETTI NON REGISTRATI` con il nome del file e del concetto mancante (sezione 5). Risolve la maggior parte dei casi.

Cerco un Concetto-paese e non lo trovo in “Luoghi” → I nomi di stato sono paese, non luogo: controlla nella sezione “Paesi” dell'indice (sezione 5).

Build fallisce dicendo che manca ai_prose → Il pezzo non dichiara la notazione dell'intervento AI (sezione 1). Non ci sono eccezioni di data. Se il codice è «nessun intervento», va scritto `ai_prose: "00"` **fra virgolette**: senza, YAML lo legge come il numero 0 e il messaggio di errore te lo dice.

Una pagina inglese mostra la navigazione italiana (o viceversa) → La cornice non dipende da `lang`: ma dall'URL: `isENSurface` è vero solo sotto `/en/` e sotto `/lab/` (sezione 8). Una pagina inglese messa altrove prende la cornice italiana.

Lo switch di lingua porta alla home invece che alla pagina gemella → Manca `translation` nel `frontmatter`, o è dichiarato su una sola delle due pagine: serve su entrambe (sezione 8).

Il banco di misura ha la testata sfasata o il piede strano → Una regola CSS dello strumento è finita fuori da `.bm` (sezione 9).

Il piano dei regimi resta bianco → Le variabili colore dello strumento sono state spostate da `:root`: il `canvas` le legge da lì (sezione 9).

Build fallisce con errore Nunjucks → Controllare la sintassi del file `.njk` modificato. Errore più comune: tag `{% %}` non chiusi.

Pagina concetto non appare pur risultando registrata → Verificare che il nome in `concepts`: del `curated` corrisponda **esattamente** (maiuscole comprese) a un `name` in `conceptsIndex.js`. Se non corrisponde, ricomparirà nell'avviso di build (sezione 5).

Chip concetti non appaiono in un articolo → Per i `writings`: verificare che l'URL dell'articolo sia presente nell'array `articles` della voce in `conceptsIndex.js`. → Per i `curated`: verificare che il campo `concepts`: sia presente e non vuoto.

Un tag di un curated non genera nessuna pagina /tag/... → Non è in `curatedTagAliases.js` (sezione 2): è decorativo, non un Argomento. Aggiungilo alla mappa se merita di diventarlo.

Netlify build fallisce → Guardare il log su `app.netlify.com` → Deploy → log del deploy fallito. Di solito è un errore di sintassi in un file `.md` o `.njk`, oppure (da oggi) un avviso di concetti non registrati che però non blocca il deploy — è solo un promemoria, non un errore.

Grafo non compare → Il grafo (pagine concetto e /mappa/) è visibile solo su schermo $\geq 900\text{px}$. Su mobile è sostituito dalla lista articoli, invariata.

12. Linea editoriale e criteri di rifiuto

Il sito funziona come una casa editrice con un solo autore: catalogo (`writings`), collane (serie), selezione con nota del curatore (`curated`), apparato critico (concetti, indice), R&D (Lab). E come ogni casa editrice, si definisce con i suoi no. Questa sezione è lo statuto: si consulta prima di pubblicare, non dopo.

La linea

Interdisciplinare per necessità, non per moda: il punto in cui editoria, epistemologia, AI e mercati si incontrano. La postura è quella dell'**attraversatore di campi**, mai del consumatore di un campo. Nessun territorio è escluso a priori (la geopolitica e l'economia sono legittime quanto l'editoria), ma ogni pezzo deve passare dal punto di attraversamento: un framework, una competenza o un'esperienza che appartengono all'autore.

I tre criteri di rifiuto

1. No ai pezzi da consumatore. Il test, per ogni `curated`: *questa chiosa poteva scriverla solo io?* Se la nota su un articolo dell'*Economist* è quella che scriverebbe qualunque lettore intelligente dell'*Economist*, non si pubblica — non perché il tema sia sbagliato, ma perché lì non passa l'autore. La firma sta nell'incrocio dei campi (l'Ucraina letta con Axelrod, l'AI letta con la semiotica), non nel tema.

2. I concetti a occorrenza singola sono cambiali. Coniare un concetto alla prima occorrenza è legittimo solo come *apertura di linea* consapevole (es. riserva cognitiva, ecologia dei media): la seconda occorrenza è pianificata. È invece un no l'*etichetta retrospettiva*: taggare come concetto una citazione di passaggio senza intenzione di tornarci — il nodo nasce morto, vicolo cieco per il lettore e contenuto thin per i crawler. Manutenzione periodica: le cambiali si onorano (secondo pezzo) o si potano (declassamento a tag o rimozione). Un catalogo è gonfio non per quanti concetti conia, ma per quante cambiali lascia scadere.

3. No ai temi dove l'esposizione costa più dell'autorità che rende. Cronaca politica interna e controversie correnti: un dirigente in carica paga quel conto senza incassarlo. Il tema caldo si tocca *col framework* (chi dissente deve dissentire dal modello, non dall'autore), mai col commento diretto da pundit. Corollario italiano: **non-adversarialità in patria** — niente posizioni contro autori di libri o firme del mercato editoriale italiano; si curano pezzi e argomenti, non persone. L'attrito intellettuale coi nomi si fa in inglese, a distanza di sicurezza professionale.

Antagonisti retorici (fonti integrative)

Il catalogo attuale pende verso l'ecosistema NYT/Atlantic/Economist: liberal-democratico, europeista, pro-humanities. Non è un difetto da nascondere ma un'asimmetria da compensare: ogni tanto va curato

l'avversario migliore — voci autorevoli con framework, che costringono a una chiosa che solo l'autore può scrivere (anche fosse “ecco dove sbaglia, e perché il modello regge lo stesso”). Chi pubblica anche chi lo smentisce non è classificabile.

Asse	Fonti	Perché
Geopolitica	John Mearsheimer; Niall Ferguson	Realismo offensivo e revisionismo storico: dissenso con modello, compatibile col criterio 1
AI	Gary Marcus; Marc Andreessen	I due lati della frizione: critica sistematica del paradigma LLM; accelerazionismo argomentato
Ecologia dei media	L.M. Sacasas (<i>The Convivial Society</i>); Jonathan Haidt (<i>After Babel</i>)	La tradizione Illich-Ellul-Postman da premesse non progressiste; media ecology empirica
Femminismo	Mary Harrington; Louise Perry	Critica femminista conservatrice della rivoluzione sessuale — il contrappunto al cluster Melandri
Economia	Tyler Cowen (<i>Marginal Revolution</i>); Byrne Hobart (<i>The Diff</i>)	Libertarismo imprevedibile; finanza tech contrarian
Lingua	John McWhorter	Linguistica eterodossa, incrocia l'asse semiotico
Italia	Il Foglio, solo versante liberale	Garantismo, cultura di mercato, europeismo non sentimentale. L'anima cattolico-reazionaria (à la Meotti) è fuori linea. Selezione pezzo per pezzo, mai per testata

Regola d'ingaggio: mai per quota, solo quando il singolo pezzo supera il criterio 1. In italiano niente firme antagoniste (criterio 3, corollario).